



Università
di Genova

DIBRIS DIPARTIMENTO
DI INFORMATICA, BIOINGEGNERIA,
ROBOTICA E INGEGNERIA DEI SISTEMI

Report - Parallelisation of Matrix Multiplication

High Performance Computing Course – University of Genova – a.a.
2025/26

Author:

Marco Castagna – S4700984

Contents

1	Head of the Analysis	1
2	Single Core Analysis	2
2.1	Hotspot Identification	2
2.2	Vectorization Analysis and Best Sequential Time	5
2.2.1	Advanced Compiler Optimizations and Best Sequential Time	6
3	OpenMP	7
3.1	First Solution Adopted	7
3.1.1	Performance Analysis	7
3.1.2	Investigating the Bottleneck	8
3.1.3	The Mathematical Proof of Cache Thrashing	8
3.2	Final Solution: Second Approach Adopted	9
3.2.1	Performance Analysis with Loop Tiling	10
3.3	Scalability	12
4	CUDA	15
4.1	Google Colab Environment	15
4.2	First Solution Adopted: Naive Implementation	18
4.3	Second Solution Adopted: Tiled Implementation with Shared Memory	21
4.4	Scalability	24
5	Conclusions	25

1 Head of the Analysis

The algorithm has a cubic time complexity of $\mathcal{O}(N^3)$, requiring approximately $2N^3$ operations and dealing with an $\mathcal{O}(N^2)$ data footprint.

Tools: I used the Intel C Compiler (icx) and the Intel Advisor GUI to perform most of the analysis.

Machine: 210 room's Machine equipped with an Intel® Core™ i9-12900K Processor. It has 24 logical processors:

- 16 physical cores
- 8 Performance cores with Hyper-Threading up to 2
- 8 Efficiency cores

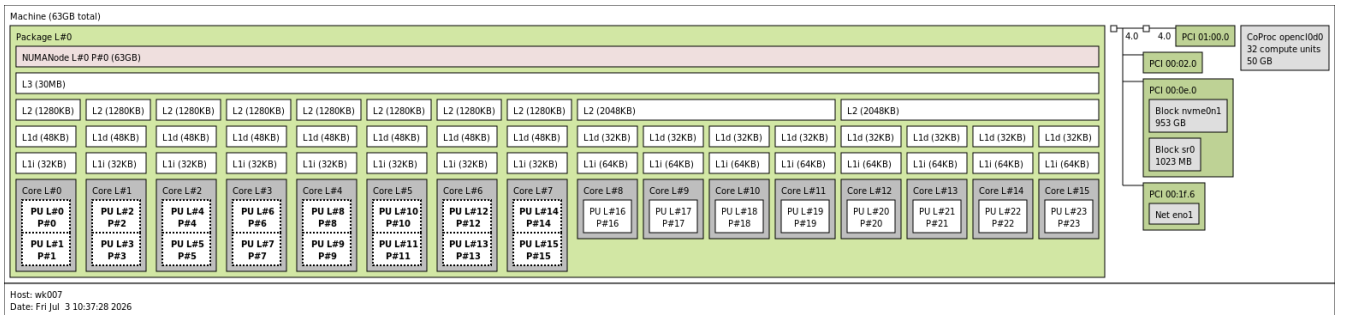


Figure 1: Hardware topology of the i9-12900K processor.

To accurately record the execution time, I decided to utilize the high-precision timers provided by the OpenMP library (`omp_get_wtime`).

The source code features a design optimization where the nested loops are inverted compared to the standard mathematical matrix multiplication formula. As discussed in class, this loop order ($i-k-j$) allows us to exploit the cache lines through both spatial and temporal locality. Although this shifts the memory stores from $\mathcal{O}(N^2)$ to $\mathcal{O}(N^3)$, the rewritten algorithm significantly saves memory read time by avoiding cache misses, ultimately increasing performance.

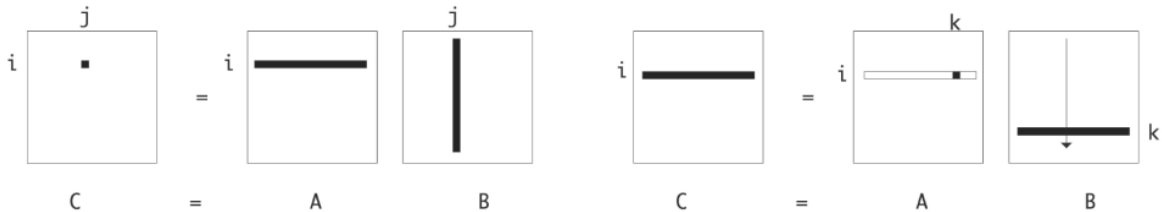


Figure 2: Optimization using the $i-k-j$ loop order.

2 Single Core Analysis

2.1 Hotspot Identification

Starting with a baseline approach, I compiled the code disabling all optimizations (`icx -g -O0 -xHost -fiopenmp -o bin/matmul mat_mul.c`) and analyzed the algorithm with Data size $N = 2000$, resulting in a computation time of 14.400218 seconds. I then decided to double the size to $N = 5000$, which yielded a computation time of 227.300751 seconds.

Since the matrix multiplication algorithm has an asymptotic complexity of $\mathcal{O}(N^3)$, we can theoretically estimate the execution time for the new dimension using this formula:

$$T_{new} = T_{known} \cdot \left(\frac{N_{new}}{N_{known}} \right)^3$$

- T_{new} : Estimated execution time for the new matrix size.
- T_{known} : Recorded execution time for the known matrix size.
- N_{new} : New matrix dimension.
- N_{known} : Known matrix dimension.

Applying our empirically measured values:

$$T_{5000} = 14.40 \cdot \left(\frac{5000}{2000} \right)^3 = 14.40 \cdot 15.625 = 225 \text{ seconds}$$

This theoretical estimate of 225 seconds closely matches our actual recorded time of 227.30 seconds.

Through performance profiling, it becomes evident that the hotspot resides at row 35 (`for (j = 0; j < n; ++j)`): this single loop consumes 99.9% of the total execution time.

By compiling with `-O0`, the compiler is forced to generate naive scalar instructions, completely bypassing the AVX vector capabilities. While Intel Advisor confirms the loop correctly processes `Float64` data, the massive execution time (≈ 227 seconds) is driven by the total lack of register allocation. Without optimizations, the CPU is forced to reload loop indices and matrix elements from the memory hierarchy at every single iteration, executing one purely scalar addition and multiplication at a time.

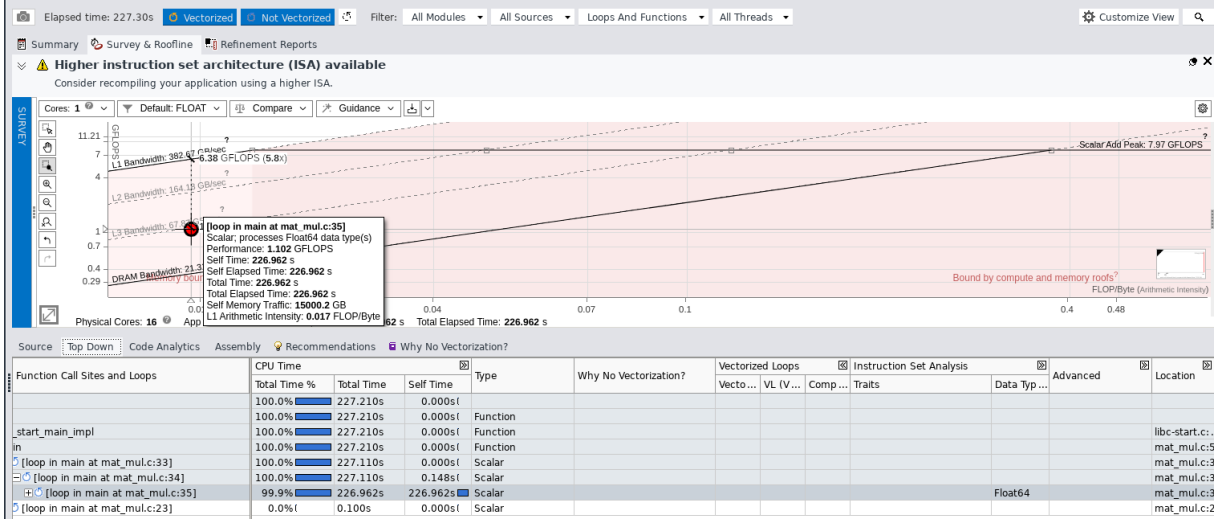


Figure 3: Roofline Model of the sequential unoptimized algorithm ($N = 5000$).

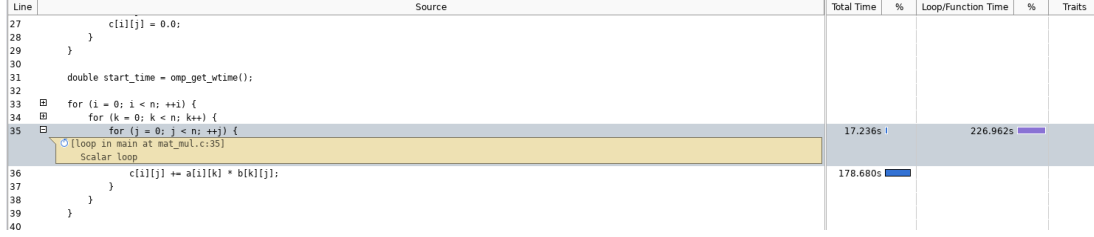


Figure 4: Hotspot identification showing the scalar loop execution.

The unoptimized algorithm is heavily memory-bound, but the Roofline Model reveals an interesting detail: the main hotspot sits just below the L3 Cache Bandwidth limit (≈ 67.78 GB/s and 1.13 GFLOPS), instead of dropping down to the slower DRAM limit. The arithmetic intensity is very low (only 0.017 FLOP/Byte), compared to the theoretical value of 0.0625 (2 FLOPs / 32 Bytes) for the core operation:

`c[i][j] += a[i][k] * b[k][j]`. This drop happens because the `-O0` flag disables register allocation. As a result, the compiler is forced to constantly reload loop variables and array pointers from the memory stack during every iteration. However, since the loop sequence is optimized (`i-k-j`), the memory access is contiguous (good spatial locality). This allows the CPU's hardware prefetcher to bring data from the RAM into the L3 cache ahead of time. Because of this, the execution is bottlenecked by the L3 cache bandwidth and the use of scalar instructions, rather than the slow DRAM latency.

Figure 6 shows the assembly code when we compile with `-O0` (no optimization). We can see that the compiler does not use the CPU registers well. Instead, it constantly reloads loop variables and matrix pointers from the stack using the `%rbp` pointer.

The biggest problem is at line `0x4013e4` with the instruction `vmovsdq %xmm0, (%rax,%rcx,8)`. This instruction takes 89.392 seconds, which is 39.34% of the total execution time. It happens because the program saves the numbers into matrix *C* at every iteration, instead of keeping them inside a fast CPU register. Writing data to the main memory (RAM) is very slow and takes a lot of time. This is why memory operations represent 53% of the total performance loss.

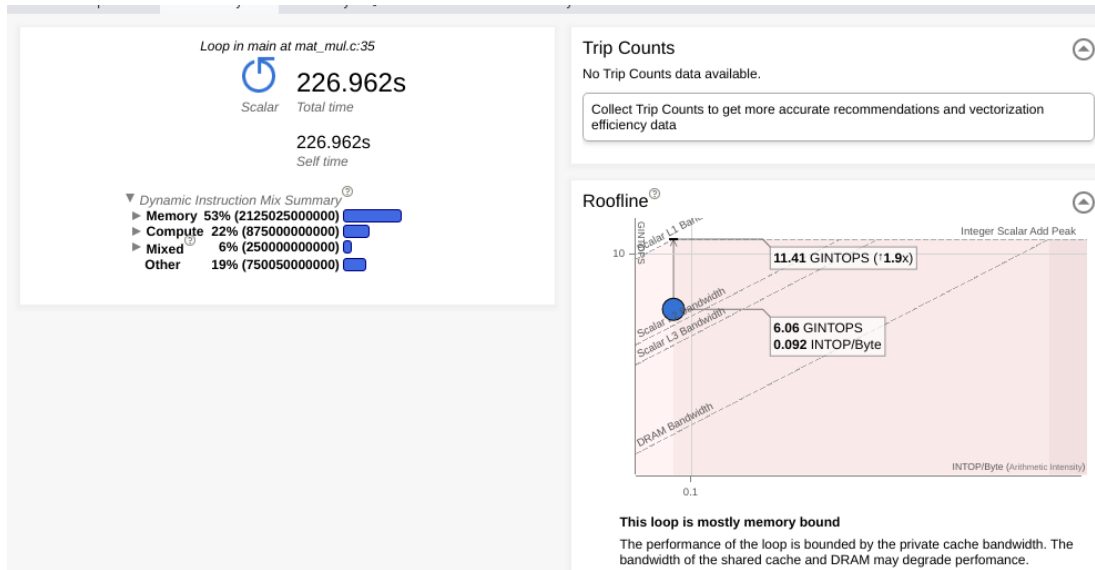


Figure 5: Code analytics detailing the memory and compute overhead.

Module: matmult0x401378				Total Time	%	Self Time	%	Traits
Address	Line	Assembly						
0x401378		Block 1: 125025000000						
0x401378	35	movl -0x1c(%rbp), %eax		0.240s		0.240s		
0x40137b	35	cmpl -0x14(%rbp), %eax		0.470s		0.470s		
0x40137e	35	jnl 0x4013f7						
0x401384		Block 2: 125000000000						
0x401384		movq -0x50(%rbp), %rdx		15.416s		15.416s		
0x401388		movq -0x58(%rbp), %rsi		0.648s		0.648s		
0x40138c		movq -0x60(%rbp), %rdi		14.982s		14.982s		
0x401390	36	movq -0x28(%rbp), %rax		0.440s		0.440s		
0x401394	36	movsxdil -0x18(%rbp), %rcx						
0x401398	36	imul %rdi, %rcx		0.048s		0.048s		
0x40139c	36	shl \$0x3, %rcx		0.132s		0.132s		
0x4013a0	36	add %rcx, %rax		0.280s		0.280s		
0x4013a3	36	movsxdil -0x20(%rbp), %rcx		14.876s		14.876s		
0x4013a7	36	vmoovsdq (%rax,%rcx,8), %xmm0		0.568s		0.568s		
0x4013ac	36	movq -0x30(%rbp), %rax		16.762s		16.762s		
0x4013b0	36	movsxdil -0x20(%rbp), %rcx		0.252s		0.252s		
0x4013b4	36	imul %rsi, %rcx		0.028s		0.028s		
0x4013b8	36	shl \$0x3, %rcx		0.072s		0.072s		
0x4013bc	36	add %rcx, %rax		0.148s		0.148s		
0x4013bf	36	movsxdil -0x1c(%rbp), %rcx		0.304s		0.304s		
0x4013c3	36	vmulsdq (%rax,%rcx,8), %xmm0, %xmm0		15.128s		15.128s		
0x4013c8	36	movq -0x38(%rbp), %rax		36.926s		36.926s		
0x4013cc	36	movsxdil -0x18(%rbp), %rcx		0.520s		0.520s		
0x4013d0	36	imul %rdx, %rcx		0.016s		0.016s		
0x4013d4	36	shl \$0x3, %rcx		0.080s		0.080s		
0x4013d8	36	add %rcx, %rax		0.072s		0.072s		
0x4013db	36	movsxdil -0x1c(%rbp), %rcx		0.332s		0.332s		
0x4013df	36	vaddsdq (%rax,%rcx,8), %xmm0, %xmm0		2.304s		2.304s		
0x4013e4	36	vmoovsdq %xmm0, (%rax,%rcx,8)		89.392s		89.392s		
0x4013e9	35	movl -0x1c(%rbp), %eax		16.178s		16.178s		
0x4013ec	35	add \$0x1, %eax		0.216s		0.216s		
0x4013ef	35	movl %eax, -0x1c(%rbp)		0.028s		0.028s		
0x4013f2	35	jmp 0x401378 <Block 1>		0.104s		0.104s		

Figure 6: Assembly code generated with the -O0 flag.

2.2 Vectorization Analysis and Best Sequential Time

To fix the memory bottleneck and the slow scalar execution from the baseline, I used the compiler’s advanced optimizations for the 210 room machine’s architecture.

Compilation command: `icx -g -O3 -xHost -fiopenmp -qopt-report=3 -o bin/matmul mat_mul.c`

Data size: $N = 5000$

Execution Time: 9.592203 seconds

Using the `-O3` and `-xHost` flags reduced the execution time by a factor of $\approx 24\times$. To understand this massive speedup, I analyzed the vectorization report (`-qopt-report=3`). Because the nested loops are perfectly arranged (`i-k-j`) to respect spatial locality, the compiler successfully vectorized the innermost loop (`j`). The code was compiled using **AVX2** instructions with a physical vector length of 4 (packing four 64-bit double variables into a 256-bit register). Furthermore, the compiler applied aggressive **Loop Unrolling** (processing 8 elements per iteration) to keep the execution units fully saturated.

Running Intel Advisor on this optimized program confirms the report and shows a completely transformed Roofline Model:

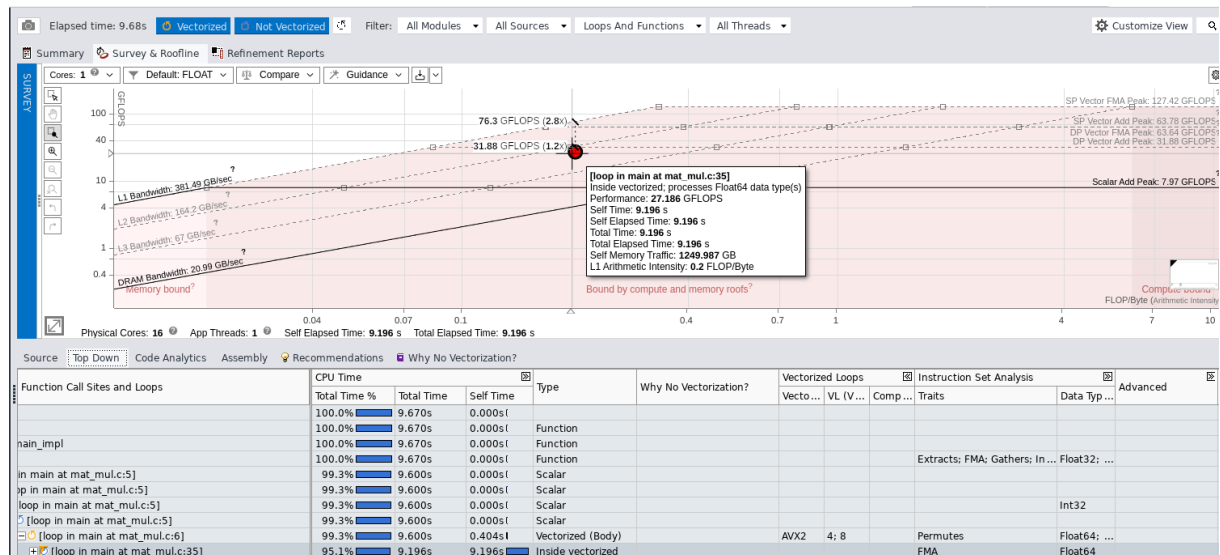


Figure 7: Roofline Model of the optimized, vectorized sequential algorithm ($N = 5000$).

- **Instruction Set and FMA:** The hotspot transitioned from slow *Scalar Float64* execution to *Vectorized AVX2*. Crucially, it now leverages **FMA** (Fused Multiply-Add) hardware units, performing addition and multiplication simultaneously in a single hardware step.
- **Arithmetic Intensity:** The L1 Arithmetic Intensity improved significantly to **0.200 FLOP/Byte**. This proves that the variables are being efficiently reused within the fast CPU registers and L1 cache, drastically reducing the slow memory reloads.
- **Compute Performance:** The hotspot moved dramatically higher on the graph, peaking at **27.18 GFLOPS** (up from just 1.13 GFLOPS in the scalar baseline). This proves the bottleneck has shifted away from memory latency and is now fully exploiting the compute capabilities of the single physical core.

2.2.1 Advanced Compiler Optimizations and Best Sequential Time

After setting a strong baseline with `-O3` and `-xHost`, I decided to increase the problem size from $N = 5000$ to $N = 10000$ and test the advanced compiler flags to maximize sequential performance:

- **-xHost**: Tells the compiler to use the best instructions for the specific CPU running the code (enabling AVX2/FMA).
- **-ffast-math**: Relaxes strict floating-point rules, allowing the compiler to reorder math operations for faster execution.
- **-ipo (Interprocedural Optimization)**: Analyzes the whole program at once to optimize how data flows between functions.
- **-fno-alias**: Tells the compiler that the arrays (`a`, `b`, `c`) do not overlap in memory. This removes slow safety checks.

To measure the impact of these flags, I ran the algorithm with $N = 10000$ using different combinations:

Compiler Flags	Execution Time
<code>-O3 -xHost</code> (Baseline)	76.061213 seconds
<code>-O3 -xHost -ffast-math</code>	76.023915 seconds
<code>-O3 -xHost -ipo</code>	79.507378 seconds
<code>-O3 -xHost -fno-alias</code>	76.115624 seconds
<code>-O3 -xHost -ipo -ffast-math -fno-alias</code>	79.374703 seconds

Table 1: Impact of advanced compiler flags on execution time ($N = 10000$).

Analysis of the Results

Increasing the matrix dimension to $N = 10000$ means the static memory footprint of each matrix is roughly 800 MB, resulting in about 2.4 GB of total RAM allocation. At this massive scale, the baseline `-O3 -xHost` configuration provided the best sequential time.

Breaking down the lack of improvement from the advanced flags:

- **-ipo**: This flag did not improve performance because all our core logic resides exclusively within the `main` function, making the optimization completely unnecessary.
- **-ffast-math**: Since our algorithm only relies on a basic multiply-add operation, there are no complex mathematical functions (like square roots or transcendental functions such as exponentials and logarithms) for the compiler to simplify or “cheat” on. The hardware FMA is already doing the absolute minimum work possible.
- **-fno-alias**: This flag tells the compiler the matrices do not overlap. However, the compiler is already smart enough to generate clean, vectorized AVX2 code without it.

Therefore, the baseline compilation command (`icx -g -O3 -xHost -fiopenmp -o bin/matmul_mat_mul.c`) perfectly applies vectorization without over-complicating the memory access, yielding our **Best Sequential Time of 76.06 seconds**.

3 OpenMP

To compile the parallel version of the program, I used the following command:

```
icx -g -O3 -xHost -fiopenmp -o bin/matmul_p mat_mul_parallel.c
```

3.1 First Solution Adopted

I decided to insert the directive `#pragma omp parallel for default(none) shared(a, b, c, n) private(j, k) schedule(dynamic)` on the first of the three nested loops:

```
#pragma omp parallel for default(none) shared(a, b, c, n) \
    private(j, k) schedule(dynamic)
for (i = 0; i < n; ++i) {
    for (k = 0; k < n; k++) {
        for (j = 0; j < n; ++j) {
            c[i][j] += a[i][k] * b[k][j];
        }
    }
}
```

With this configuration, the threads divide the execution of the outermost loop (`i`). This means that every time a thread takes a single iteration `i` of the first loop, it runs the two inner loops entirely, which equals $N \times N$ iterations. I chose this approach to avoid **race conditions** and to allow each thread to process its portion of data without interference.

Since the workstation has a **hybrid architecture** (with different types of cores), I selected the **dynamic scheduler**.

If I had used the **static scheduler** with $N = 10000$ and 24 threads, the workload would be divided equally from the start: each thread would get exactly $10000/24 \approx 417$ iterations of the first loop. However, by using the dynamic scheduler, the workload division is managed at runtime by OpenMP. The distribution is not perfectly equal from the beginning; instead, each thread requests and processes a new block of work as soon as it becomes available.

I experimented multiple times by running the exact same code with both scheduling policies. The results show that executions with the dynamic scheduler are always slightly faster. This indicates that the static scheduler creates a **load imbalance** and extra overhead on the efficiency cores, which have smaller caches.

For example, with $N = 10000$:

- **Static Scheduling:** 34.022240 seconds
- **Dynamic Scheduling:** 31.731761 seconds

3.1.1 Performance Analysis

With an execution time of **36.67 seconds** for $N = 10000$, a major doubt immediately arises. I am highly skeptical about using 24 threads and only dropping from 76 seconds (sequential) to 37 seconds (parallel). This represents a very poor speedup.

Looking at the Roofline model, further questions appear:

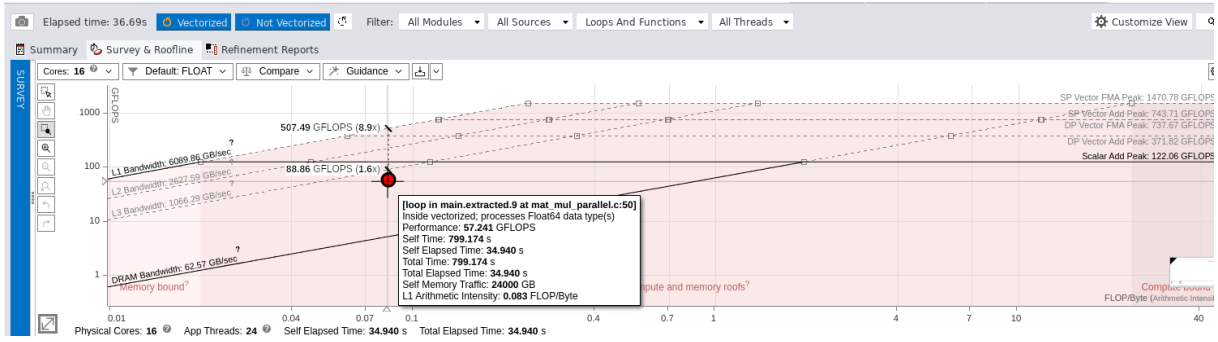


Figure 8: Roofline Model of the first parallel execution ($N = 10000$).

The system seems to be stuck in the exact same initial situation, where performance strictly depends on memory speed (**Memory-Bound**).

3.1.2 Investigating the Bottleneck

- **Could this be caused by False Sharing?** Reasoning about the structure, false sharing should not be an issue here. Each thread works on its own independent row i , meaning threads do not overwrite or interfere with each other's data cache lines.
- **Could it be a NUMA effect?** The answer is no. The workstation architecture does not feature separate RAM nodes for different sockets; the entire physical RAM is shared equally by all cores.

Therefore, the real issue must be related to **cache misses**. We can classify cache misses into three categories:

1. **Compulsory Misses:** Occur when data is read for the first time. These are unavoidable.
2. **Conflict Misses:** Caused by mapping conflicts or False Sharing. (Excluded).
3. **Capacity Misses:** This is the real problem.

3.1.3 The Mathematical Proof of Cache Thrashing

Let's analyze the memory footprint for $N = 10000$. Consider a single thread executing a single iteration of the outermost loop i (for example, Thread 0 handles $i = 0$). This thread must execute the remaining two nested loops (k and j) entirely, from 0 to 9999.

The core accumulation formula is:

$$c[i][j] += a[i][k] * b[k][j]$$

To calculate a single row of matrix C , the thread requires:

Row i of Matrix A consists of 10,000 double elements, which take up about **80 KB**. It fits easily within the L1d or L2 cache of the core, but the formula requires us also to iterate $b[k][j]$, so the entire **Matrix B** from top to bottom. A 10000×10000 matrix of double values requires **800 MB** of memory.

An 800 MB memory footprint completely floods the L1d, L2, and even the shared L3 cache of the processor.

Conclusion

While the CPU possesses massive computing power, the enormous volume of concurrent data requests completely chokes the memory bus. Every thread demands a quantity of data that cannot physically fit into its cache.

As a result, parallelism is severely degraded because all 24 threads are forcing simultaneous access to the physical RAM, constantly overwriting the shared L3 cache (**Cache Thrashing**). The threads spend roughly 90% of their execution time idle, waiting for the completely saturated memory bus to deliver data from the RAM.

3.2 Final Solution: Second Approach Adopted

A well-known solution to break the Memory Wall is to rewrite the algorithm by splitting the data into smaller blocks that fit perfectly into the cache hierarchy. This can be achieved through techniques such as *Loop Tiling*, Cache-Aware Architectures (like the GotoBLAS approach), or Cache-Oblivious Algorithms (as described in the research papers in the slides).

To implement this, I chose to apply Loop Tiling by nesting additional loops. This technique splits the memory space into smaller, bounded tiles to prevent cache saturation. More importantly, it increases the task granularity, allowing the cores to achieve maximum performance. While this is not the ultimate optimization and more advanced tuning is possible, my primary goal is to break the Memory Wall created by the previous algorithm.

```
#pragma omp parallel for default(none) shared(a, b, c, n, BLOCK_SIZE) \
    schedule(dynamic)
for (int i = 0; i < n; i += BLOCK_SIZE) {
    for (int k = 0; k < n; k += BLOCK_SIZE) {
        for (int j = 0; j < n; j += BLOCK_SIZE) {

            // Boundary Computation prevents Segmentation Faults
            int i_end = (i + BLOCK_SIZE > n) ? n : i + BLOCK_SIZE;
            int k_end = (k + BLOCK_SIZE > n) ? n : k + BLOCK_SIZE;
            int j_end = (j + BLOCK_SIZE > n) ? n : j + BLOCK_SIZE;

            // Inside the Cache block
            for (int ii = i; ii < i_end; ++ii) {
                for (int kk = k; kk < k_end; ++kk) {
                    // This loop will be vectorized in AVX2/FMA by the compiler
                    for (int jj = j; jj < j_end; ++jj) {
                        c[ii][jj] += a[ii][kk] * b[kk][jj];
                    }
                }
            }
        }
    }
}
```

```

    }
}

```

We can analyze this implementation using the **PCAM Methodology** (Partitioning, Communication, Agglomeration, Mapping) studied during the course:

1. Partitioning

I applied **domain decomposition**. Instead of treating the 10000×10000 matrices as massive, monolithic blocks, I partitioned the data domain by introducing internal loops. The matrix space is split into small 64×64 sub-matrices (tiles), which serve as the fundamental unit of computation.

2. Communication

The algorithm is designed to achieve an ideal scenario: *embarrassingly parallel computation* with no need for communication. The problem is decomposed so that tasks can run concurrently without sharing data. By assigning independent horizontal strips of Matrix C to each thread, I avoided memory collisions. This eliminated the need for **locks**, **barriers**, or expensive collective operations like **reductions**. Every thread operates in total isolation within its cache.

3. Agglomeration

Instead of passing individual 64×64 tiles directly to OpenMP (which would create a granularity that is too fine and generate massive scheduling overhead), I **agglomerated** the workload. I placed the `#pragma omp parallel for` directive only on the outermost loop `i`. This packs entire horizontal bands (comprising 64 rows by 10,000 columns for $N = 10000$) into a single macro-task. This strategy ensures a very high computation-to-overhead ratio, allowing the threads to perform calculations continuously for several milliseconds (coarse-grained parallelism).

4. Mapping

The mapping phase is delegated directly to the runtime scheduler, following a master-slave/worker paradigm. This is managed by configuring OpenMP with the `schedule(dynamic)` policy, which helps balance the workload across the cores at runtime.

3.2.1 Performance Analysis with Loop Tiling

On the `wk007` workstation (Lab 210) using 24 OpenMP threads, the execution time for the tiled algorithm drops significantly:

Computation time (N=10000, BLOCK=64): 8.602647 seconds

This demonstrates a clear and substantial speedup compared to the untiled parallel version, proving that our cache-blocking strategy successfully bypassed the memory bottleneck.

This performance leap is highly visible when examining the updated Roofline model:

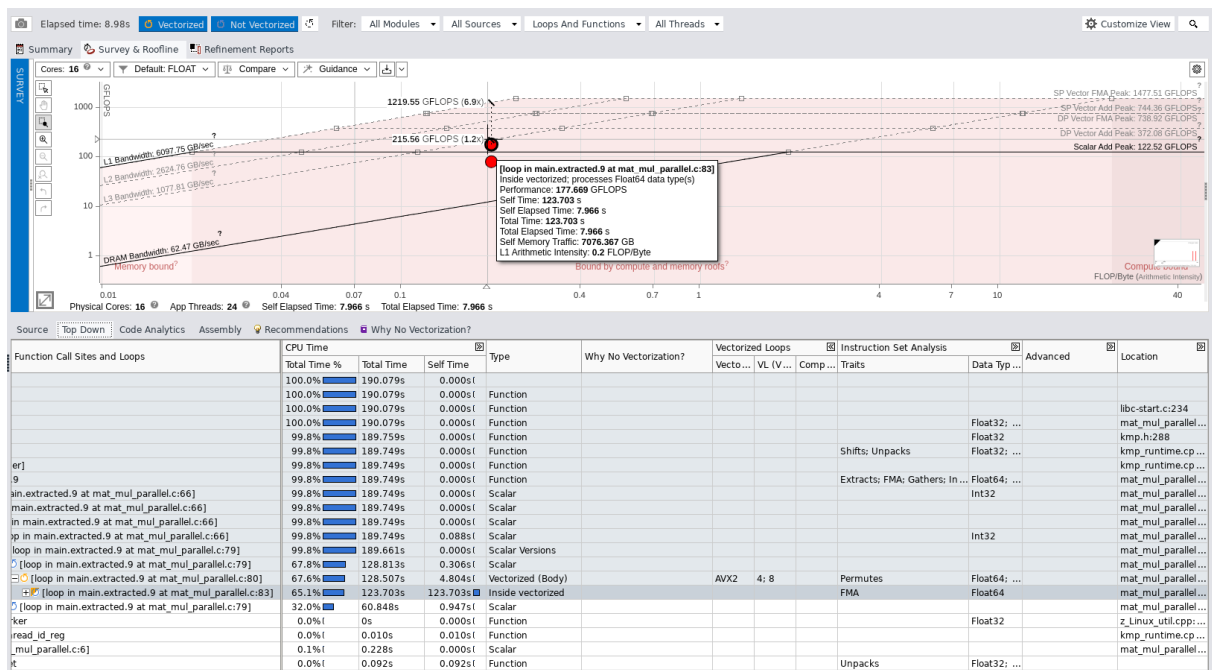


Figure 9: Roofline Model of the tiled parallel execution ($N = 10000$, BLOCK=64).

The computational throughput reaches an impressive **177 GFLOPS**, a massive increase compared to the ≈ 57 GFLOPS recorded in the previous parallel run without tiling.

However, even with this great result, the graph shows that the program is still not using the maximum power of the CPU when all cores are active. To better understand this behavior and identify remaining overheads, a deeper investigation into thread scalability and problem size scaling is required. So, I evaluated the performance across three different matrix sizes— 5000×5000 , 10000×10000 , and 15000×15000 —to systematically analyze both **Speedup** and **Parallel Efficiency**.

3.3 Scalability

Graphs generated by the `benchmark.sh` script:

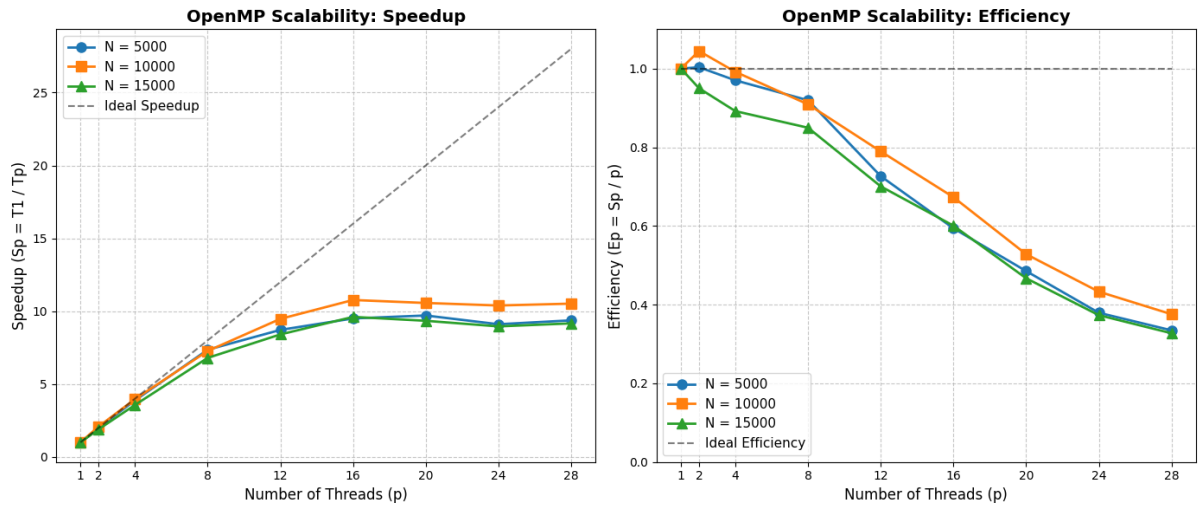


Figure 10: Speedup and Efficiency analysis across different thread counts and problem sizes.

What can we say? In all runs, the speedup is linear up to 8 threads, which are very likely the Performance Cores (P-Cores). When moving to the Efficiency Cores (E-Cores), the situation changes and the curve flattens out when reaching 16 threads. Beyond that point, the logical Hyper-Threading threads are used, entering Simultaneous Multithreading (SMT). In general, these threads are great for a computer, but in our algorithm, they do not give us any extra speedup or high parallelism. In fact, the theoretical formulas for speedup and efficiency give us a very clear picture of the computer architecture; the efficiency clearly stays very high on the P-Cores.

This performance drop of the E-Cores creates a sort of implicit barrier. This means that even if the faster threads finish their task, they literally stand still waiting for the slower ones, creating a **load imbalance**. I try to mask this problem with dynamic scheduling, but this overhead is always present. One solution that comes to mind is to distribute the workload unequally: giving much more to the Performance Cores and less to the Efficiency Cores, maybe using a different block size for them. I notice that they share the L2 cache, which shows me that they fill up their memory much faster than the others. Therefore, it is normal that with all 16 physical cores, the curve flattens out when half of the cores in the system are slower and have less capacity.

Intel Advisor confirms this:

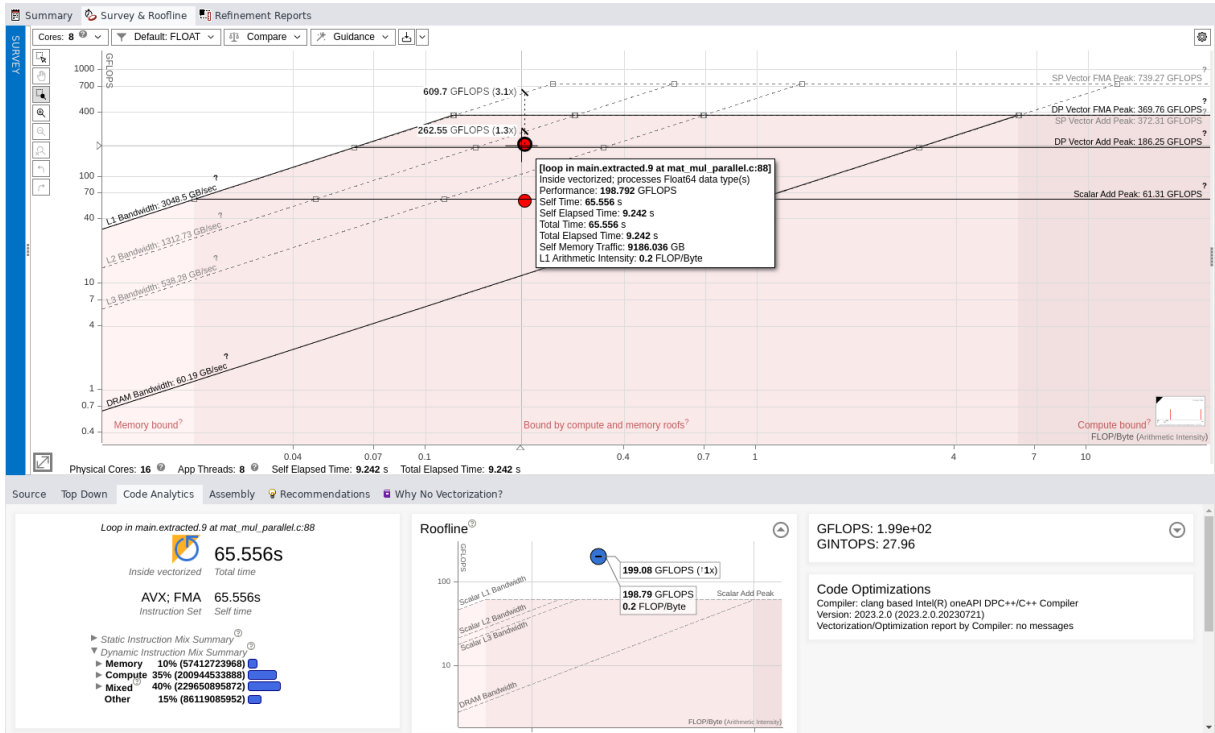


Figure 11: Roofline Model of the tiled algorithm running strictly on 8 P-Cores.

where the 8 cores perform better than all 16 together, reaching almost 200 GFLOPS and hitting the intrinsic computing limit of the machine!

Looking at the images, the most optimal problem size seems to be the one with matrices of $N = 10000$, where the speedup and efficiency are better compared to the other sizes.

In conclusion, I experimented with different block sizes to see how they fit into the cache and to find the best configuration for this algorithm on this hardware. I tested blocks of 32×32 , 64×64 , and 128×128 on the various problem sizes used before:

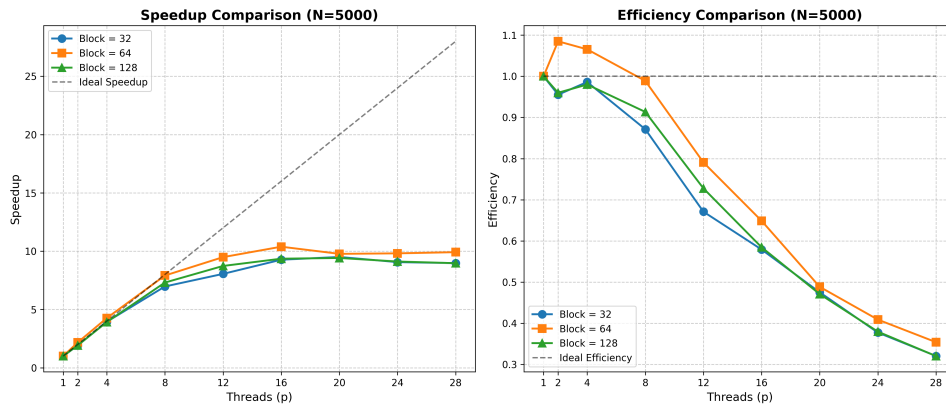


Figure 12: Speedup and Efficiency comparison for different block sizes at $N = 5000$.

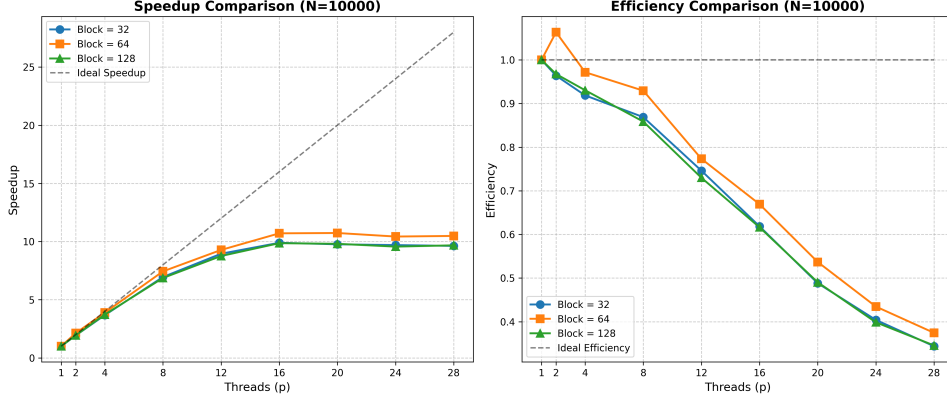


Figure 13: Speedup and Efficiency comparison for different block sizes at $N = 10000$.

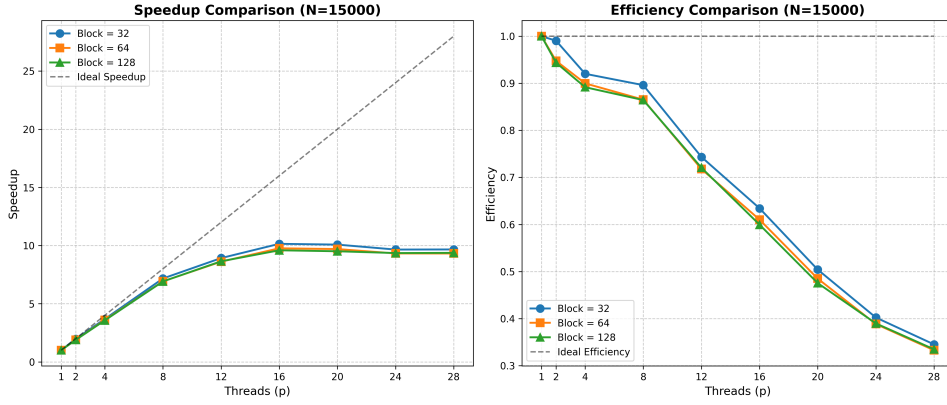


Figure 14: Speedup and Efficiency comparison for different block sizes at $N = 15000$.

The clear winner remains the 64×64 size, but there is something interesting when we move to larger N . In the last image with $N = 15000$, the algorithm with the smaller blocks performed the best compared to the others. This is quite strange but makes it very interesting. It makes me think that as the problem grows, proper decomposition has a big impact on performance, meaning that we must build the algorithm strictly based on the volume of data.

Regarding the experiments, I conclude that the parallel computation achieved a good speedup. In these empirical tests, we crash directly into theory, where reaching perfect performance is incredibly difficult. To optimize everything in the best way, it is not enough to make things cache-optimized; we must also consider core heterogeneity, which heavily complicates the aspect of load balancing.

4 CUDA

4.1 Google Colab Environment

To evaluate the GPU performance, the tests were migrated to the Google Colab environment. First, I analyzed the CPU architecture provided by the platform to establish a sequential baseline for comparison.

CPU architecture for the sequential test:

```
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:         46 bits physical, 48 bits virtual
Byte Order:            Little Endian
CPU(s):                2
On-line CPU(s) list:   0,1
Vendor ID:             GenuineIntel
Model name:            Intel(R) Xeon(R) CPU @ 2.20GHz
```

To compile the sequential baseline, I used the following command:

```
!gcc -O3 -march=native -fopenmp matmul.c -o matmul_seq
```

- **Computation time ($N = 5000$):** 91.53734 seconds
- **Computation time ($N = 10000$):** 764.730147 seconds
- **Computation time ($N = 15000$):** 2733.177186 seconds

Note: The situation is different with the remote CPU, which is significantly slower in single-core performance compared to the local workstation.

GPU Device Properties

The core of this phase is the GPU provided by Colab. Using the device query, I extracted the hardware specifications of the assigned Tesla T4:

```
--- Device 0: "Tesla T4" ---
  CUDA Capability Major/Minor version:    7.5
  Total amount of global memory:          14.56 GBytes
  Number of multiprocessors (SMs):        40
  Total amount of constant memory:        65536 bytes
  Total amount of shared memory per block: 49152 bytes
  Total number of registers available per block: 65536
  Warp size:                             32
  Maximum threads per multiprocessor (SM): 1024
  Maximum threads per block:              1024
  Max dimension size of a thread block (x,y,z): 1024, 1024, 64
  Max dimension size of a grid size (x,y,z): 2147483647, 65535, 65535
```

From the data retrieved from the device, we can observe the presence of 40 Streaming Multiprocessors (SMs) and the maximum number of threads per block, which guides us on how to structure the thread blocks in our code. Furthermore, the global memory of 14.56

GB is more than sufficient to allocate the test matrices up to $N = 15000$. Investigating the Tesla T4 model, I verified that the GPU is based on compute capability version 7.5.

NVIDIA T4 Specifications

Performance	Tensor Core Turing 320 NVIDIA CUDA Core 2,560 Single Precision Performance (FP32) 8.1 TFLOPS Mixed precision (FP16/FP32) 65 FP16 TFLOPS INT8 Accuracy 130 INT8 TOPS INT4 Accuracy 260 INT4 TOPS
Interconnection	Gen3 PCIe x16
Memory	CAPACITY GDDR6 16 GB Bandwidth 320+ GB/s
Food	70 watts

Figure 15: NVIDIA Tesla T4 Specifications.

Looking at the performance specifications, the architecture features Tensor Cores. These are ideal for matrix multiplications because they can compute an entire matrix product in a single clock cycle, making them excellent for neural networks.

I also observed a total of 2560 NVIDIA CUDA cores divided among the 40 SMs, meaning each SM contains 64 CUDA cores. However, the specifications only showcase the performance for Single Precision (FP32), Mixed Precision (FP16/FP32), INT8, and INT4. There is no explicit mention of Double Precision (FP64) performance, which is critical

for our specific problem since our matrix multiplication relies entirely on double-precision variables.

According to the vendor specifications, the GPU is based on the NVIDIA Turing architecture. Analyzing the documentation in the *NVIDIA Turing Architecture Whitepaper*, the structure of an SM is detailed as follows:



Figure 16: Turing Streaming Multiprocessor (SM) Architecture.

In this diagram, I do not see any cores explicitly dedicated to FP64 operations. Looking more closely at the documentation, I noticed that:



Note: The TU102 GPU also features 144 FP64 units (two per SM), which are not depicted in this diagram. The FP64 TFLOP rate is 1/32nd the TFLOP rate of FP32 operations. The small number of FP64 hardware units are included to ensure any programs with FP64 code operates correctly.

Figure 17: Turing Architecture FP64 throughput limitations.

Consequently, the primary execution units of this GPU are optimized for FP32, whereas FP64 support is included only for compatibility purposes and operates at significantly reduced performance. Specifically, the FP64 throughput is limited to 1/32 of the FP32 TFLOP rate. Therefore, while FP32 performance reaches 8.1 TFLOPS, the FP64 performance drops to approximately 0.25 TFLOPS (254.4 GFLOPS), relying on only 80 units in total (2 per SM).

This GPU appears to be designed primarily for AI inference and computer graphics, rather than scientific High-Performance Computing (HPC). For scientific workloads, alternative enterprise GPUs such as the A100, V100, or H100 would be far more appropriate, as they maintain a much higher FP64-to-FP32 ratio of 1:2. However, the Google Colab free tier limits our experimental setup strictly to the Tesla T4.

Ultimately, because there are only 2 FP64 ALUs available per SM to be shared among the 32 threads of a warp, executing a single FP64 operation requires 16 clock cycles!

4.2 First Solution Adopted: Naive Implementation

The first strategy involves using two-dimensional thread blocks that are multiples of the warp size (32 threads) to ensure full warp utilization. Initially, a block size of 16×16 was chosen, creating a large two-dimensional grid of blocks.

In this algorithm, each thread is responsible for computing a single element of the result matrix C using the classic row-by-column multiplication. Consequently, each thread must determine its respective row and column indices. It iterates through all the columns of a given row in matrix A , and all the rows of a specific col in matrix B , accumulating the sum of the products. Furthermore, contiguous memory cells are assigned to adjacent threads to maximize efficiency by exploiting **Memory Coalescing**.

Each thread calculates its own global position within matrix C using the global two-

dimensional grid of blocks. Logically, this creates a massive grid equivalent to the size of C , which is divided into logical blocks that are scheduled across the various SMs in parallel. Every thread within these blocks obtains a global `row` and `col` coordinate using the following formulas:

- `row = blockIdx.y * blockDim.y + threadIdx.y` (where `.y` denotes the row position)
- `col = blockIdx.x * blockDim.x + threadIdx.x` (where `.x` denotes the column position)

This mechanism allows each thread to identify exactly which row and column it needs to traverse. To construct a precise grid of blocks, the grid dimensions are calculated as $(n / blockDim.x, n / blockDim.y)$. Since the block count must be an integer, a padding factor of `blockDim - 1` is added to handle the ceiling rounding. For instance, with a 16×16 block, the formula becomes $((n + 15) / 16, (n + 15) / 16)$. This approach ensures that matrix boundaries are correctly included even if N is not perfectly divisible by 16.

Once the partitioning strategy is established, the computation proceeds with the standard matrix multiplication. Within the GPU, this calculation must be managed using a monodimensional memory layout. The core idea is to leverage this 1D representation so that each thread can retrieve the necessary elements from matrices A and B . This requires a single loop running up to N .

To access the global memory for matrix A , the thread's `row` is multiplied by N to locate the correct row offset, to which the loop index `k` is added. For matrix B , the access must jump from row to row; therefore, index `k` is multiplied by N at each iteration, and the thread's global `col` index is added to target the correct column.

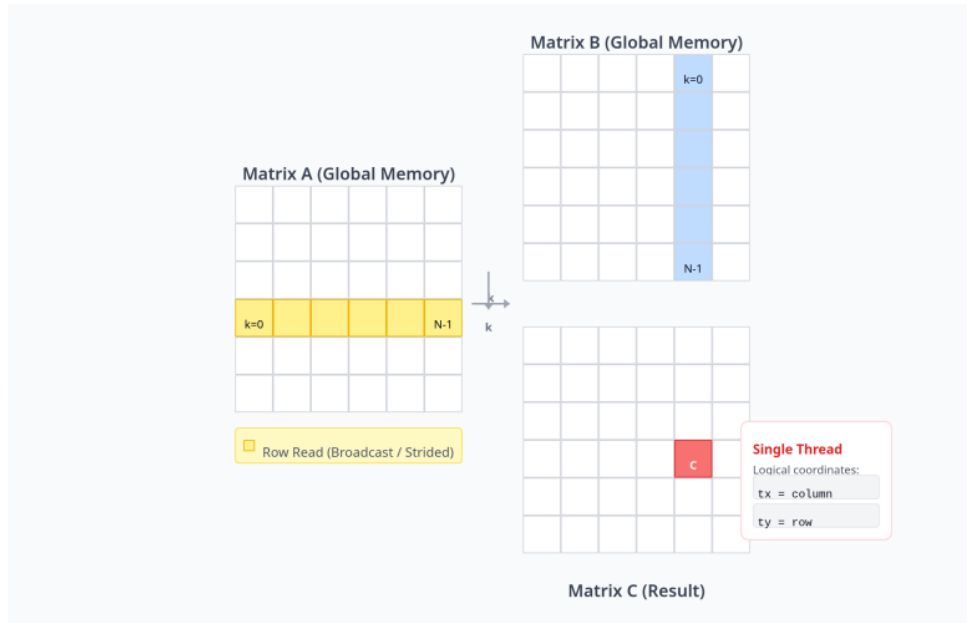


Figure 18: Memory access pattern for the naive CUDA implementation.

```

__global__ void matMulKernel(double *a, double *b, double *c, int n) {

    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;

    // Check that the thread is within the matrix boundaries
    if (row < n && col < n) {
        double sum = 0.0;

        // 1D array simulating a 2D matrix (row * n + col)
        for (int k = 0; k < n; k++) {
            sum += a[row * n + k] * b[k * n + col];
        }
        c[row * n + col] = sum;
    }
}

```

Thus, the parallelization is achieved easily. A test run with $N = 10000$ yields the following results:

```

Matrix Dimension N           : 10000
Transfer Time CPU -> GPU      : 340.948517 ms
GPU Kernel Computation Time   : 8916.573242 ms
Transfer Time GPU -> CPU      : 167.988388 ms
Total Time (Data + Compute)   : 9425.509766 ms

```

This results in a total execution time of approximately 9.5 seconds, achieving a speedup of about $81\times$ compared to the sequential CPU baseline.

Through performance profiling, it becomes evident that the primary delay is not caused by the CPU-to-GPU data transfer (PCIe bottleneck), but rather by the kernel execution itself. Using `nvprof`, the data indicates that 90.67% of the total time is consumed by the computation: GPU activities: 90.67% 1.26170s `matMulKernel(double*, double*, double*, int)`. Furthermore, as the dimension N increases, the percentage of time spent in the kernel grows correspondingly, reaching 94% for $N = 10000$ and 97% for $N = 15000$.

A more in-depth profiling session using the visual profiler, NVIDIA Nsight Systems, on a run with $N = 10000$ confirms this behavior:

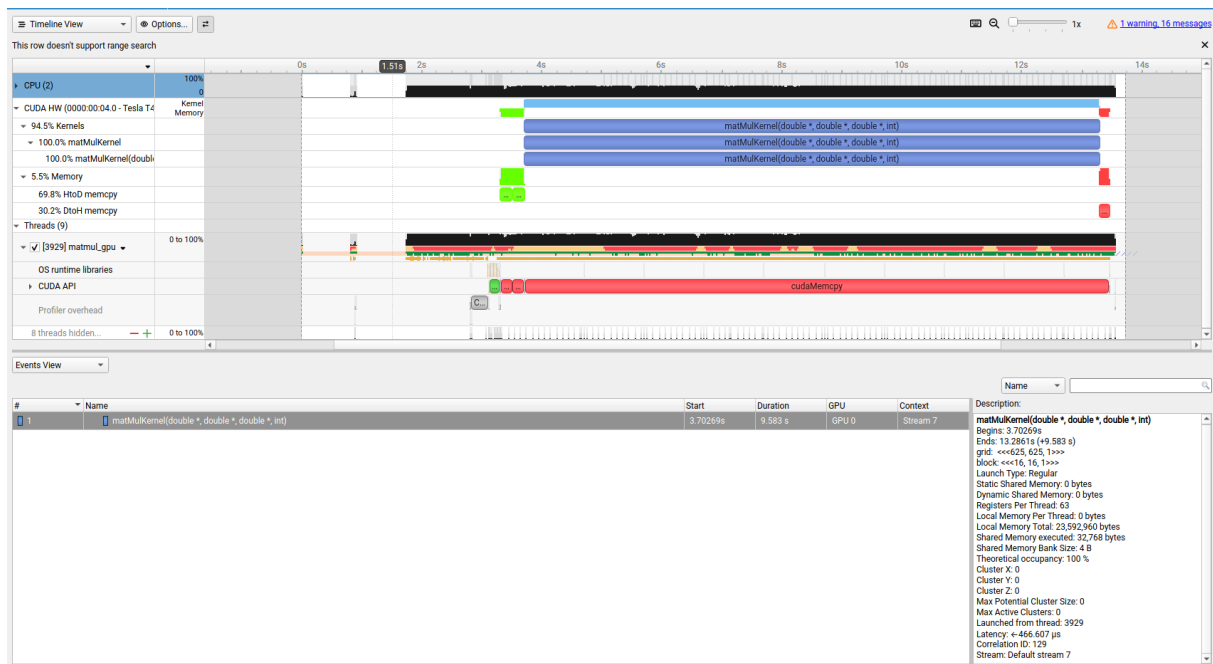


Figure 19: NVIDIA Nsight Systems timeline for the naive CUDA kernel ($N = 10000$).

The timeline clearly shows that the vast majority of the execution time is dedicated to the kernel. However, a critical observation from the profiler is the complete absence of Shared Memory utilization.

To put this into perspective:

- Accessing **Global Memory** costs between 400 and 800 clock cycles.
- Accessing **Shared Memory** requires only 2 to 4 clock cycles.

Undoubtedly, the algorithm can be optimized. In its current naive state, each thread repeatedly fetches every single element of matrices A and B directly from the extremely slow Global Memory N times to compute the dot product. This makes the algorithm inevitably inefficient.

Although the implementation benefits from **Memory Coalescing**, which consolidates memory accesses into fewer transactions per warp (given that threads in our setup request contiguous memory elements based on their `row` and `col` coordinates), it completely lacks **Temporal Locality**. The L1 and L2 caches are too small to retain the massive amounts of data required. Consequently, once a block finishes its computation, the subsequent adjacent blocks may request the exact same values, but they are forced to re-read them entirely from Global Memory. This repeated fetching from high-latency memory strictly limits the performance, creating a severe **Memory-Bound** bottleneck.

4.3 Second Solution Adopted: Tiled Implementation with Shared Memory

To limit the use of Global Memory as much as possible, it is necessary to coordinate the threads of the block using Shared Memory itself. To calculate an element of matrix C , the

idea is to proceed by “tiles”, similarly to what I did with OpenMP, rather than moving entire rows or columns at a time.

As for the agglomeration phase, in OpenMP the calculation of a “strip” of C (made of many small pieces next to each other) was assigned to a single thread. In CUDA, I apply a similar concept, but using an entire block of threads, with the goal of calculating not just a single element, but a complete tile of C . Consequently, I advance this block of threads exactly over the tiles of matrices A and B of the same size as the block, moving to the right for A and downwards for B .

To do this, each block allocates two portions of Shared Memory dedicated exclusively to those threads (of the same size as the tiles), into which the values of A and B needed for the calculation will be inserted. These areas work like a personal cache to use at each iteration. The coordination of the algorithm is divided into two distinct phases, separated by synchronization barriers (`__syncthreads()`):

For each logical tile t :

- **Phase 1 (Reading):** Each thread of the block reads two values from the Global Memory, relating to its local position in the tile (ty , tx) and, above all, to its position in the global grid (row , col). Considering that the sliding of A happens on the same row and that of B on the same column, for the current tile t each thread takes two values and saves them in the dedicated areas in Shared Memory. In this way, the two shared tiles are filled with a single cooperative read (each thread loads exactly one value per matrix).
- **Phase 2 (Calculation):** All threads perform the calculations using only their private registers and reading **exclusively** from the Shared Memory. Each thread calculates the dot product by multiplying the values of its row in Shared- A by those of its column in Shared- B .

With this approach, reads from the very slow Global Memory are reduced to only two for each thread at each logical step, followed by a single final write of the result in C at the end of the iterations.

```
__global__ void matMulKernel(double *a, double *b, double *c, int n) {
    __shared__ double As[TILE_SIZE][TILE_SIZE];
    __shared__ double Bs[TILE_SIZE][TILE_SIZE];

    int tx = threadIdx.x;
    int ty = threadIdx.y;
    int col = blockIdx.x * TILE_SIZE + tx;
    int row = blockIdx.y * TILE_SIZE + ty;

    double sum = 0.0;

    for (int t = 0; t < (n + TILE_SIZE - 1) / TILE_SIZE; t++) {
        if (row < n && t * TILE_SIZE + tx < n)
            As[ty][tx] = a[row * n + t * TILE_SIZE + tx];
        else
            As[ty][tx] = 0.0;
```

```

    if (t * TILE_SIZE + ty < n && col < n)
        Bs[ty][tx] = b[(t * TILE_SIZE + ty) * n + col];
    else
        Bs[ty][tx] = 0.0;

    __syncthreads();

    for (int k = 0; k < TILE_SIZE; k++) {
        sum += As[ty][k] * Bs[k][tx];
    }
    __syncthreads();
}

if (row < n && col < n) {
    c[row * n + col] = sum;
}
}

```

Results

BlockSize: 16x16
 H2D Time : 358.513092 ms
 Kernel Time : 8225.050781 ms
 D2H Time : 170.397049 ms
 Total Time (Data + Compute) : 8753.960938 ms

This tiled approach achieves a speedup of approximately $87\times$, which is a measurable improvement over the $81\times$ speedup of the naive version. Furthermore, the profiler data confirms that Shared Memory is now actively utilized. The register pressure per thread has also decreased significantly (dropping from 63 down to 42 registers per thread).

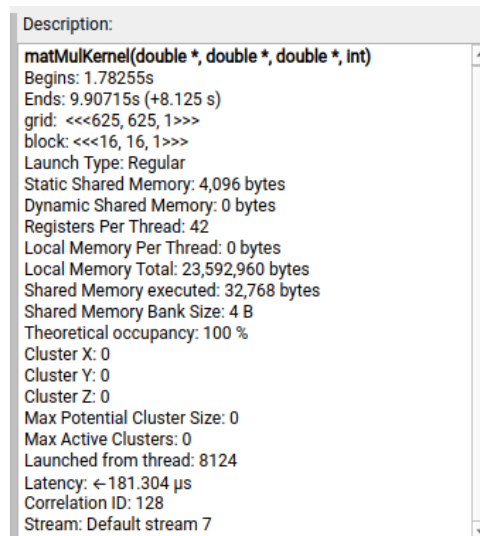


Figure 20: NVIDIA Nsight the Tiled Shared Memory kernel.

4.4 Scalability

In CUDA, we cannot set the total number of threads directly; we can only choose the block size. Because of this, I decided to test the speedup and efficiency using different block sizes. Specifically, I tested 8×8 , 16×16 , and 32×32 blocks (the maximum limit of the hardware) for different matrix sizes: $N = 5000$, $N = 10000$, and $N = 15000$.

Analysis of the Results:

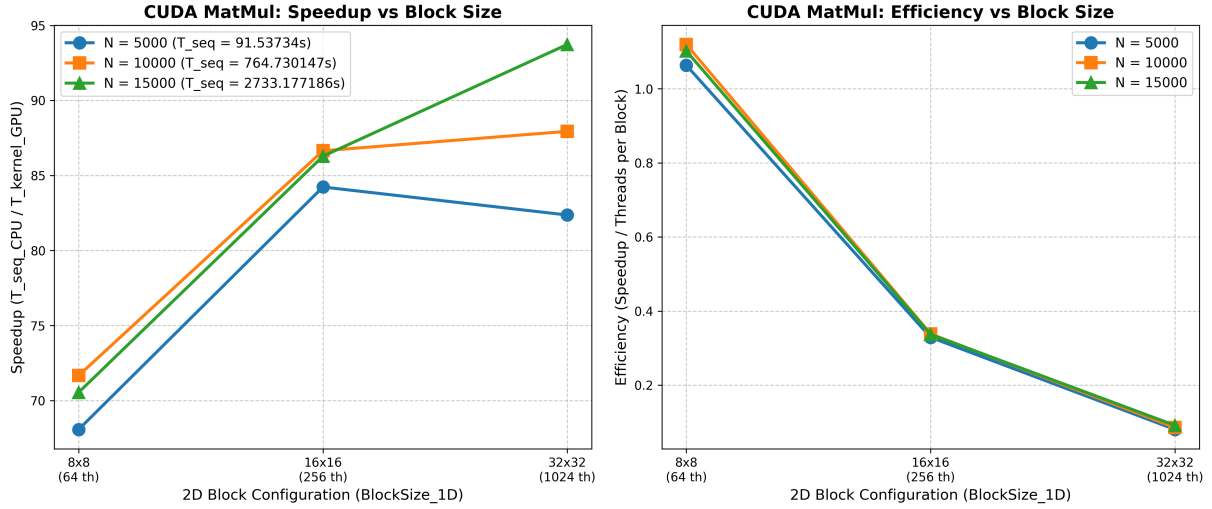


Figure 21: Speedup and Efficiency analysis for different CUDA block sizes.

- The 8×8 block (64 threads):** This setup exhibits the lowest speedup across all tested matrix sizes. With 64 threads per block and $N = 5000$, the system must schedule a massive grid of $625 \times 625 = 390,625$ blocks. This results in about 10,000 blocks assigned to each SM, which creates significant scheduling overhead. In a compute-bound scenario limited by a small number of FP64 units, managing such a high number of blocks slows down the hardware scheduler. Furthermore, kernel profiling revealed that the 8×8 configuration leads to an excessive number of memory operations. Because the tile size is small, the algorithm requires more iterations to cover the entire matrix. Each iteration performs two global memory reads per thread and a synchronization barrier (`__syncthreads()`). Consequently, the increase in total tile iterations significantly raises memory latency and synchronization overhead. On the other hand, this setup is highly efficient per thread; this is expected, as small blocks use fewer registers, allowing the active threads to work without hitting resource limits.
- The 16×16 block (256 threads):** This setup is a great balance (the “sweet spot”) for all matrix sizes. It finds the best compromise between filling the SMs (occupancy) and reducing the scheduling overhead.
- The 32×32 block (1024 threads):** This huge block size is only better for very large matrices ($N = 15000$). For smaller matrices, there are too few blocks to keep the SMs fully busy (low occupancy). Because of this, there are not enough warps to do fast context switches, so the hardware cannot hide the memory delay (latency). Also, a block of 1024 threads uses too many registers, which can cause

register spilling (moving data to slower memory). The big drop in the efficiency graph clearly shows this hardware limit.

5 Conclusions

In conclusion, CUDA is definitely a more flexible and modern tool for parallel computing. However, in my case, the Tesla T4 has severe limits for double-precision (FP64) math. Because of this, the execution times are very similar to the OpenMP results on the CPU. Even though we are comparing two completely different architectures, the power of modern CPU cores shows that they are still very strong for parallel work and should not be ignored.

Overall, this project taught me a lot. It helped me look past simple parallelization and understand the many hardware factors that can ruin performance, like memory limits, high register use, and scheduling overhead. Using the profiling tools from the course was very important to analyze and understand these problems.

Finally, for the future, it would be very interesting to test the code using single-precision (`float` - FP32) numbers. This simple change would let us use all the 2560 standard CUDA cores on this GPU, or even the Tensor Cores. This would certainly bring a huge improvement in overall performance.